

7. 运行、可观测性与风险控制

OpenViking 的关键处理链路大量使用异步队列和后台任务，因此运行质量取决于模型服务、向量库、文件系统、队列状态和任务注册表的协同。技术说明和实施交付中应重点关注以下运维点。

关注项	指标/现象	处理建议
资源入库耗时	wait_complete 超时、 queue.wait.duration_ms 偏高	检查 Parser 耗时、LLM 并发、向量库写入和模型限流
摘要失败	SemanticProcessor re-enqueue 或 document status failed	区分临时模型错误和永久错误，必要时调低并发或修正文档格式
检索质量	召回过宽、答案引用不准、score 过低	检查 L0/L1 摘要质量、rerank 配置、 target_uri 和 context_type
记忆污染	重复记忆、过时偏好、不准确案例	启用去重合并，保留归档审计，定期做热度和低质记忆治理
权限边界	跨账号或跨用户检索异常	检查 RequestContext、owner_space、 account_id 和 HTTP 身份头
增量同步	旧内容未删除或新内容未索引	检查 target_uri、生命周期锁、watch task 和 diff 同步日志

实施建议

先以客服机器人或投标资料库作为高价值场景验证闭环，再把同一知识底座扩展到更多 AI 应用。每扩展一个场景，优先复用资源入库、记忆沉淀、检索增强和权限治理能力，只新增场景 Prompt、工具链和业务流程编排。

8. 源码依据与模块地图

模块	关键文件	说明
总体架构文档	docs/zh/concepts/01-architecture.md	服务层、处理层、双层存储和数据流说明
上下文层级	docs/zh/concepts/03-context-layers.md	L0/L1/L2 分层设计
资料抽取	docs/zh/concepts/06-extraction.md	Parser、TreeBuilder、SemanticQueue 和多格式支持
检索设计	docs/zh/concepts/07-retrieval.md	find/search、TypedQuery、分层检索和重排
会话与记忆	docs/zh/concepts/08-session.md	Session commit、归档、记忆抽取和去重
多租户	docs/zh/concepts/11-multi-tenant.md	account/user/agent 隔离和身份边界
资源 API	openviking/server/routers/resources.py	add_resource、temp_upload、knowledge documents
搜索 API	openviking/server/routers/search.py	find、search、grep、glob 路由
文档树落位	openviking/parse/tree_builder.py	临时目录、URI 解析、唯一命名
语义处理	openviking/storage/queuefs/ semantic_processor.py	L0/L1 生成、增量更新、熔断重试、向量化
会话实现	openviking/session/session.py	消息、归档、异步提交和 active_count